Universidad
Complutense & Politécnica
de Madrid

Master's in Formal Methods in Computer Science

Master's Thesis

Type Theory and Categorical Models
A Unified Approach

Miguel Laredo Barbadillo
Raúl Lara Cabrera
Guillermo Román Díez

Madrid, August 10, 2025

This Final Degree Project has been submitted to the ETSI Informáticos of the Universidad Politécnica de Madrid for its defense.

Este Trabajo Fin de Grado se ha depositado en la ETSI Informáticos de la Universidad Politécnica de Madrid para su defensa.

Master's in	Formal Methods in Computer Science
Title	Type Theory and Categorical Models: A Unified Approach

Author	Miguel Laredo Barbadillo
Tutors	Raúl Lara Cabrera
	Dpto. Sistemas Informáticos
	Guillermo Román Díez
	Dpto. Lenguajes y Sistemas Informáticos e Ingeniería de Softwaree

Escuela Técnica Superior de Ingenieros Informáticos
Universidad Politécnica de Madrid

Contents

1 λ-Calculus **7**
1.1 Untyped λ-Calculus 8
1.1.1 The Language of λ-Calculus 8
1.1.2 Equivalence of terms 9
1.1.2.1 Subterms 9
1.1.2.2 Free Variables 10
1.1.2.3 Variable Substitution 11
1.1.2.4 α-equivalence 13
1.1.2.5 β-equivalence 14
1.1.2.6 η-equivalence 16

Abstract

This work investigates the interplay of lambda calculus, type theory, category theory, computation, and functional programming through the Curry-Howard-Lambek correspondence. It explores lambda calculus as a computational foundation, its expression in type-safe functional programming, and its categorical semantics, highlighting how these fields unify to enhance program design.

Esta trabajo examina la interacción entre el cálculo lambda, la teoría de tipos, la teoría de categorías, la computación y la programación funcional a través de la correspondencia Curry-Howard-Lambek. Explora el cálculo lambda como base computacional, su expresión tipada y su semántica categórica, destacando cómo estos campos se unifican para mejorar el diseño de programas.

Keywords: Lambda Calculus, Type Theory, Category Theory, Computation, Functional Programming, Curry-Howard-Lambek Correspondence

Chapter 1

λ-Calculus

In the early 20th century, mathematicians such as Bertrand Russell, David Hilbert, and Kurt Gödel were trying to set complete, consistent, and decidable foundations for mathematics. Within this context, Alonzo Church, in his effort to formalize the notion of computability, introduced a minimal symbolic language based upon function abstraction and function application, this system is now called λ-calculus.

In 1936, Church used lambda calculus to address Hilberts Entscheidungsproblem –whether a mechanical method could determine the truth of any first-order logic statement. He proved no such algorithm exists, establishing the undecidability of first-order logic.

One of the nuances formalized by the λ-calculus is the distinction between extensional and intensional equality. The extensional approach to equivalence states that two functions are equivalent if they share input output pairs for every possible input. Its intensional counterpart extends this notion of equality by requiring that the procedures that compute these pairs share complexity i.e. they take the same steps toward yielding a result.[1]

The Lambda Calculus together with Type Theory lay the theoretical groundwork that shaped the design of languages like LISP, Haskell and Erlang to name a few. More advanced type systems than those present in general purpose programming languages enable computers to verify mathematical statements and produce rigorous proofs.

[1] Let p be a sufficiently large prime, and let $f, g : \mathbb{Z}_p \to \mathbb{Z}_p$ be defined by $f(x) = x^2$ and $g(x) = \log_a(a^{x+2})$, where $a \in \mathbb{Z}_p^\times$ is a fixed primitive root. Although f and g are extensionally equal, i.e., they yield the same output for all inputs in $\mathbb{Z}_p$, they are intensionally distinct. The function f performs a simple squaring operation, while g requires evaluating a discrete logarithm, intractable in general. (Nadal 2021)

1.1 Untyped λ-Calculus

In order to get acquainted with the λ-calculus, let us develop a simple example to familiarize ourselves before we begin with a more formal approach to this discipline. Consider the function $f(x) = x + 1$, the most straightforward way to express this using λ-calculus would be $(\lambda x.x + 1)$, where the lambda denotes that x is being captured and used as a parameter to perform some computation. Evaluating $f(2)$ using this newly created lambda term would look like this: $(\lambda x.x + 1)(2) \rightsquigarrow (2 + 1) \rightsquigarrow 3$. This process of reducing a λ-expression is referred to as β-reduction, it will be covered formally later in the chapter.

1.1.1 The Language of λ-Calculus

Definition 1.1.1. The set of all lambda terms Λ is defined inductively as follows:

- (Variable) If $x \in \mathcal{V}$, then $x \in \Lambda$.
- (Abstraction) If $x \in \mathcal{V}$ and $M \in \Lambda$, then $(\lambda x.M) \in \Lambda$.
- (Application) If $M, N \in \Lambda$, then $(MN) \in \Lambda$.

Where $\mathcal{V} = \{x, y, z, ...\}$ represents a countably infinite set of variable names. These notions are conveniently synthesized into the following abstract syntax:

$$\Lambda := \mathcal{V} \mid \lambda x.\, x \mid \Lambda\,\Lambda$$

that directly translates into the following Haskell snippet:

```
data Term = Var String
          | Abs String Term
          | App Term Term
```

The key takeaway of this definition is that abstraction and application when paired with a reduction procedure, capture the meaning of function in a way that is abstract yet useful, and enables computation.

Notation 1. We use uppercase letters, e.g. M, N, L, P, Q to denote elements of Λ and lowercase e.g. x, y, z to denote elements of $\mathcal{V}$.

Remark 1. From the definition above we extract that $\lambda x.x + 1 \notin \Lambda$, given this circumstance, the need to find an accurate representation for numbers within this definition arises. See Definition **??**

Example 1.1.1. Some examples of valid λ terms generated using the grammar in 1.1.1: y, $(\lambda x.(xx))$, $(\lambda x.(\lambda y.x))$, $(((\lambda x.(xy))(\lambda y.y))z)$. The Haskell counterpart for the second term is:

```
Abs "x" (Abs "y" (Var "x"))
```

As a convention to avoid notational cluttering, the outermost parenthesis can be omitted e.g. $\lambda x.x$ is read as $(\lambda x.x)$. Also, application is left-associative and binds tighter than abstraction, so MNP is parsed as $(MN)P$ and $\lambda x.MN$ means $\lambda x.(MN)$, not $(\lambda x.M)N$. Likewise, abstraction is right-associative e.g. $\lambda x.\lambda y.M$ is $\lambda x.(\lambda y.M)$ and binds more weakly than application. Haskell Curry introduced what we now referr to as currying, its main purpose is to avoid unnecessary lambdas. Below is a summary of common notational conventions used throughout the text:

- $\lambda x.\, x \iff (\lambda x.\, x)$
- $M\,N\,P \iff (M\,N)\,P$
- $\lambda x.M\,N \iff \lambda x.(M\,N)$
- $\lambda x.\lambda y.f(x,y) \iff \lambda xy.f(x,y)$

All these are just to keep notation straight-forward, the formal definition does not leave precedence under-specified thanks to parenthesis. Those interested in the implementation need not worry, as Lean4 enforces an explicit order on constructors.

1.1.2 Equivalence of terms

Now that the λ-calculus language has been introduced, it is time we move on to the mechanics of computation, manifested through β-equivalence. α and η equivalence will also be introduced. These two are more difficult to grasp. Informally, it could be said that α-equivalence is equivalence programs up to renaming of local variables. In this same scenario, η-equivalence would singnify equivalence of programs up to wrapping or unwrapping them with an identity function.

1.1.2.1 Subterms

From the ground up, we begin define the concept of subterm, since the definition of the language is self-referential and terms might contain other terms.

Definition 1.1.2. Given $T \in \Lambda$, $\mathbf{Sub} : \Lambda \to \mathscr{P}(\mathscr{V})$ maps a term to the set of it's subterms.

$$\begin{aligned}
&\mathbf{Sub}(x) = \{x\} \\
&\mathbf{Sub}(MN) = \mathbf{Sub}(M) \cup \mathbf{Sub}(N) \cup \{MN\} \\
&\mathbf{Sub}(\lambda x.M) = \mathbf{Sub}(M) \cup \{\lambda x.M\}
\end{aligned}$$

Where $x \in \mathscr{V}$ and $M, N \in \Lambda$. In Haskell:

```
subTerms :: Term -> Set.Set Term
subTerms (Var x) = Set.singleton (Var x)
subTerms (App m n) =
    Set.insert (App m n) (Set.union (subTerms m) (subTerms n))
subTerms (Abs x m) =
    Set.insert (Abs x m) (subTerms m)
```

Remark 2. Let $\preceq$ be the subterm relation on λ-terms, defined by

$$M \preceq N \iff M \in \mathbf{Sub}(N)$$

$\preceq$ is a partial order relation. This statement is follows intuitively from the definition, it can be proven using induction on the derivation tree.

Definition 1.1.3. A term $S \in \mathbf{Sub}(M)$, $M \in \Lambda$ is said to be proper if $S \neq M$. $\mathbf{Sub}(M) \setminus \{M\}$ would be the set of proper subterms.

Definition 1.1.4. Let $M = \lambda x.N$ be a λ-term:

- The variable x is the binding variable of the abstraction $\lambda x.N$.
- An occurrence of a variable x in M is bound if $x \in \mathbf{Sub}(\lambda x.N)$.
- Free variables are those that are not bound.

Example 1.1.2. Take $M \equiv \lambda x.\, x\, y$, in this example, the x in $\lambda x.$ is binding, y is free, and x is bound. This means that y is used literally, and that x, as denoted by $\lambda x.$ is abstracted and thus subject to being replaced within the context of said abtraction. A subexample might make this explicit: $(\lambda x.\, x\, y)\, M \rightsquigarrow_\beta M\, y$. The y remained while the x was used as a placeholder for M.

Note 1. This taxonomy is key to understanding concepts such as α-equivalence intuitively.

1.1.2.2 Free Variables

The set of Free Variables **FV**, those that will not get replaced during the process of reduction, is computed using:

Definition 1.1.5. Given $T \in \Lambda$, $\mathbf{FV} : \Lambda \to \mathscr{P}(\mathscr{V})$ outputs the set of free variables for T:

$$\begin{aligned}
&\mathbf{FV}(x) = \{x\} \\
&\mathbf{FV}(MN) = \mathbf{FV}(M) \cup \mathbf{FV}(N) \\
&\mathbf{FV}(\lambda x.M) = \mathbf{FV}(M) \setminus \{x\}
\end{aligned}$$

Where $x \in \mathscr{V}$ and $M, N \in \Lambda$, and $\mathscr{P}$ denotes the power set. In Haskell:

```
freeVars :: Term -> Set.Set String
freeVars (Var x) = Set.singleton x
freeVars (App m n) = Set.union (freeVars m) (freeVars m)
freeVars (Abs x m) = Set.delete x (freeVars m)
```

Example 1.1.3. Computing the free variables of $\lambda x.\lambda y.yxz$. The term well-formed, so we proceed:

$$\begin{aligned}\mathbf{FV}(\lambda x.\lambda y.yxz) &= \mathbf{FV}(\lambda y.yxz) \setminus \{x\} \\ &= \mathbf{FV}(yxz) \setminus \{x, y\} \\ &= \{x, y, z\} \setminus \{x, y\} \\ &= \{z\}\end{aligned}$$

Of course, z is the only free variable in the expression as it is the only variable that is not captured by some λ-abstraction.

Definition 1.1.6. A term $M \in \Lambda$ is closed if and only if $\mathbf{FV}(M) = \emptyset$, closed lambda terms are often referred to as combinators. Λ^0 is the set of all closed lambda terms.

Note 2. They are called combinators since having no free variables means that all variables are bound, and thus, a closed term just combines bound variables.

1.1.2.3 Variable Substitution

We approach the definition for α-equivalence to this end we have to introduce what it means to substitute a variable.

Definition 1.1.7. Capture-Avoiding Substitution Rules Let $M, N \in \Lambda$ and $x, y, z \in \mathcal{V}$. The substitution $M[x := N]$ is defined inductively as:

$$\begin{aligned}
x[x := N] &= N \\
y[x := N] &= y && y \neq x \\
(M_1\, M_2)[x := N] &= (M_1[x := N])\,(M_2[x := N]) \\
(\lambda x.M)[x := N] &= \lambda x.M \\
(\lambda y.M)[x := N] &= \lambda y.(M[x := N]) && y \neq x, y \notin \mathbf{FV}(N) \\
(\lambda y.M)[x := N] &= \lambda z.(M[y := z][x := N]) && y \neq x, y \in \mathbf{FV}(N)
\end{aligned}$$

where $z \notin \mathbf{FV}(M) \cup \mathbf{FV}(N)$. Implementing the latter in Haskell requires a bit more code than previous definitions, along with making some practical tradeoffs:

```
capAvoidSubs :: Term -> String -> Term -> Term
capAvoidSubs same@(Var y) x n
  | y == x = n
  | otherwise = same
capAvoidSubs (App t1 t2) s t3 =
  App (capAvoidSubs t1 s t3) (capAvoidSubs t2 s t3)
```

```
capAvoidSubs (Abs y m) x n
  | y == x = (Abs y m)
  | otherwise =
      if Set.member y (freeVars n)
      then
        let z = freshVar y (Set.union (freeVars m) (freeVars n))
        in (Abs z (capAvoidSubs (capAvoidSubs m y (Var z)) x n))
      else
        (Abs y (capAvoidSubs m x n))
  where
    freshVar :: String -> Set.Set String -> String
    freshVar s terms =
      if Set.member (s ++ "'") terms
      then
        freshVar (s ++ "'") terms
      else
        s ++ "'"
```

Note 3. `freshVar` generates a new name for binding variables such that $z \notin \mathbf{FV}(M) \cup \mathbf{FV}(N)$. In this instance we have chosen to add single quotes as long as the variable name is contained in $\mathbf{FV}(M) \cup \mathbf{FV}(N)$.

Hopefully the meaning of the first three rules is clear, they ancapsulate renaming in the context of variables and applications. The subsequent rules are about abstractions, rule four states that renaming a binding variable has no effect, renaming only modifies a term whenever the renamed variable is different from the binding variable. Rule five is the general case, that is, when the substitution just moves in together with the abstracted term. Then there is the last case, when the binding variable is a bound occurrence in the term we are substituting, which poses a problem, to see why, let us work through an example:

Example 1.1.4. If we mistakenly substitute x by $(\lambda w.\, w\, y)$ in $(\lambda y.\, y\, x)$ using rule five, we get:

$$(\lambda y.\, y x)[x := (\lambda w.\, w\, y)] = (\lambda y.\, y\, (\lambda w.\, w\, y))$$

This is problematic since the free variable y became bound after the rename. Free and bound variables should mantain their freedom or their boundedness after the rename.

Note 4. This is essentially why it is called Capture Avoiding Substitution and not simply Substitution.

After introducing rule six, we rename the binding variable to avoid the name clash:

$$(\lambda y.\, y x)[x := (\lambda w.\, w\, y)] = (\lambda z.\, z\, (\lambda w.\, w\, y))$$

The purpuse of the binding variable is to serve as a placeholder, therefore renaming does not affect the integrity of the expression, the variable gets replaced anyway when we perform a reduction.

In the previous example a bound variable is renamed to avoid a naming collision and avoid binding a previously free variable, α-equivalence is the generalization of this technique, it defines equivalence up to renaming of binding variables.

One notable result regarding substitution is the substitution lemma:

Lemma 1.1.1. Let M, N, P be lambda terms and x, y be variables. Then:

$$M[x := N][y := P] = \begin{cases} M[x := N[y := P]], & \text{if } x = y, \\ M[y := P][x := N[y := P]], & \text{if } x \neq y \text{ and } y \notin \mathrm{FV}(N). \end{cases}$$

Proof. structural induction on M. □

Notation 2. In many of the upcoming definitions inference rules are used, here is a simple introduction for those unfamiliar with the concept:

$$\frac{P_1 \quad P_2 \quad \cdots \quad P_n}{C}\ \textsc{Name}$$

where P_i are judgments assumed to hold, C is the judgment inferred from the premises and NAME is an optional identification label.

1.1.2.4 α-equivalence

Definition 1.1.8. Two lambda terms $M, N \in \Lambda$ are α-equivalent, written $M =_\alpha N$, if they are structurally identical except for the names of bound variables. Formally:

$$\frac{}{\lambda x.L =_\alpha \lambda y.L[x := y]}\ \textsc{Alpha} \qquad \frac{L =_\alpha L'}{\lambda x.L =_\alpha \lambda x.L'}\ \textsc{Abs}$$

$$\frac{L_1 =_\alpha L_1'}{L_1\, L_2 =_\alpha L_1'\, L_2}\ \textsc{AppL} \qquad \frac{L_2 =_\alpha L_2'}{L_1\, L_2 =_\alpha L_1\, L_2'}\ \textsc{AppR}$$

α-equivalence is an equivalence relation:

(Reflexivity)	$M =_\alpha M$
(Transitivity)	$M =_\alpha N$ and $N =_\alpha P \Rightarrow M =_\alpha P$
(Symetry)	$M =_\alpha N \Rightarrow N =_\alpha M$

Example 1.1.5. A simple equivalence class and two samples of not α-equivalent pairs.

$$\lambda x.\lambda y.y =_\alpha \lambda a.\lambda b.b =_\alpha \lambda z.\lambda x.x$$

$$\lambda x.\lambda y.x \neq_\alpha \lambda x.\lambda y.y \quad \lambda x.x\,y \neq_\alpha \lambda x.x\,z$$

Remember terms are α-equivalent only when bounded names are properly substituted.

Remark 3. α-equivalence comes naturally once substitution is defined, since the names of bound variables are just placeholders that will eventually hold an argument, thus getting replaced.

Note 5. Variable naming is a nightmare. De Bruijn indexing is a way of methodically assigning numbers to variable names, indicating how many binders away a variables binder is. This system eliminates the need for α equivalence to exist.

$$\lambda x.\, x \Rightarrow \lambda.0 \quad \lambda x.\lambda y.\, x \Rightarrow \lambda.\lambda.1 \quad \lambda x.\lambda y.\, y \Rightarrow \lambda.\lambda.0$$

We are better at names than we are at numbers, which is why we will prefer using symbols over De Brujn indexing. Programs like automated theorem provers and compilers do use similar naming strategies to simplify their work.

1.1.2.5 β-equivalence

Definition 1.1.9. Single-step β-reduction $\rightsquigarrow_\beta$ is defined using the following inference rules:

$$\frac{}{(\lambda x.\, M)\, N \rightsquigarrow_\beta M[x := N]}\ \textsc{Beta} \qquad \frac{M \rightsquigarrow_\beta M'}{\lambda x.\, M \rightsquigarrow_\beta \lambda x.\, M'}\ \textsc{Abs}$$

$$\frac{M_1 \rightsquigarrow_\beta M_1'}{M_1\, M_2 \rightsquigarrow_\beta M_1'\, M_2}\ \textsc{AppL} \qquad \frac{M_2 \rightsquigarrow_\beta M_2'}{M_1\, M_2 \rightsquigarrow_\beta M_1\, M_2'}\ \textsc{AppR}$$

where $M, N \in \Lambda$. In Haskell:

```
betaRed :: Term -> Term
betaRed same@(Var x) = same
betaRed (Abs x y) = Abs x (betaRed y)
betaRed (App (Abs x y) z) = capAvoidSubs y x z
betaRed (App x y) = App (betaRed x) (betaRed y)
```

Single β-reductions can be applied succesively, which induces the following definition

Definition 1.1.10. Multi-step β-reduction. We write $M \rightsquigarrow_\beta^* N$ if and only if there exists a sequence of one-step β-reductions starting from M and ending at N.[2]

$$M \equiv M_0 \rightsquigarrow_\beta M_1 \rightsquigarrow_\beta M_2 \rightsquigarrow_\beta \cdots \rightsquigarrow_\beta M_{n-1} \rightsquigarrow_\beta M_n \equiv N$$

That is, there exists an integer $n \geq 0$ and a sequence of terms $M_0, M_1, \ldots, M_n$ such that: $M_0 \equiv M$ and $M_n \equiv N$ for all i with $0 \leq i < n$, we have $M_i \rightsquigarrow_\beta M_{i+1}$.

[2] The convention in the literature is to use $\rightarrow_\beta$ for β-reductions $\twoheadrightarrow_\beta$ for multi-step β-reductions. I prefer to use $\rightsquigarrow_\beta$ and $\rightsquigarrow_\beta^n$ since this allows me to use the superscript to note the number of steps in the process β-reduction e.g. $(\lambda f.\lambda x.\, x)\, M\, N \rightsquigarrow_\beta^2$ N. Instead of $\rightsquigarrow_\beta^n$ I use $\rightsquigarrow_\beta^*$ whenever the number of steps is undefined.

Note 6. Multi-step $\rightsquigarrow_\beta^*$ is the reflexive transitive closure of $\rightsquigarrow_\beta$.

The process of succesively β-reducing a term leads raises the question of whether all terms can be reduced indefinitly. We know the answer to this, since for instance, variables in $\mathcal{V}$ cannot be reduced. The subsequent question is then, Do all all terms have some kind of normal form that cannot be reduced further? This is answered in Section **??**. To this end we introduce a formal definition of normal form that will allow us to gain insight into the normalization properties of the λ-calculus.

Definition 1.1.11. A λ-term M is in normal form if there is no term N such that $M \rightsquigarrow_\beta N$. Equivalently, M contains no subterm of the form $(\lambda x.P)\,Q$ that can be reduced by beta-reduction. In other words, M is irreducible under beta-reduction.

Definition 1.1.12. A reducible expression, or redex for short, is any subterm of the form $(\lambda x.P)\,Q$. Such a term can be reduced by a single beta-reduction step, resulting in $P[x := Q]$.

Definition 1.1.13. β-equivalence relation, equivalence of λ-terms through β-reductions.

$$M =_\beta N \iff M \rightsquigarrow_\beta^* N \vee N \rightsquigarrow_\beta^* M$$

Note 7. $=_\beta$ is the symetric closure of $\rightsquigarrow_\beta^*$.

We now explore a key aspect of the λ-calculus, in the jargon referred to as first-class citizenship, or high-order, a property of terms that restrains them from being exclusively classified as either function or argument. The example below sheds some light on the matter:

Example 1.1.6. High Order by example. Let us β-reduce the λ-term ABC and inspect the consequences:

$$\begin{aligned}
\overbrace{(\lambda f.\lambda y.f\,f\,y)}^{A}\overbrace{(\lambda x.x+1)}^{B}\overbrace{(2)}^{C} &\rightsquigarrow_\beta (\lambda y.(\lambda x.x+1)(\lambda x.x+1)\,y)(2)\\
&\rightsquigarrow_\beta (\lambda x.x+1)(\lambda x.x+1)(2)\\
&\rightsquigarrow_\beta^* (\lambda x.x+1)(3)\\
&\rightsquigarrow_\beta^* (4)
\end{aligned}$$

Taking a closer look, A, applies f twice to x, as a consequence, B is being applied twice to C. And so, very easily, we have implemented the $x+2$ function via a sequential application of $x+1$ to C.

Remark 4. As seen above, we can pass what we understand as conventional functions as arguments since they are treated alike. One can intuitively apreciate the computational expresivenes this brings with it, and how this syntactic-semantic homogeneity sets the λ-calculus apart from the classical set-theoretic approach to functions.

1.1.2.6 η-equivalence

The last and indeed the least important notion of equivalence. It refers to a program being equivalent to itself wrapped with an identity function:

Definition 1.1.14. Two lambda terms M and N are said to be η-equivalent, written $M =_\eta N$, if:

$$\frac{x \notin \mathbf{FV}(M)}{(\lambda x.\, M) =_\eta M}\ \textsc{Eta} \qquad \frac{M =_\eta M'}{\lambda x.\, M =_\eta \lambda x.\, M'}\ \textsc{Abs}$$

$$\frac{M_1 =_\eta M_1'}{M_1\, M_2 =_\eta M_1'\, M_2}\ \textsc{AppL} \qquad \frac{M_2 =_\eta M_2'}{M_1\, M_2 =_\eta M_1\, M_2'}\ \textsc{AppR}$$

The abstraction $\lambda x.\,(M\ x)$ performs the same operation as M, so if x is not used freely within M, the abstraction serves no purpose. This is shown in Eta, the rest of the inference rules propagate this equivalence throughout the whole term.

Note 8. Two terms are observationally equivalent if they cannot be distinguished by any context that can be papplied to them. η-equivalence can be understood as a form of weak extensionality, since it does not fully encode observational equivalence.

Add something about normalization properties and how they dont ensure termination. this is important for programming blah blah blah... This is why we introduce Types and Type Systems... This notion is similar to that of restricting the domain and codomain of a function.

Bibliography

Nadal, Pedro Bonilla (Sept. 2021). "Dependent Typing through Closed Cartesian Categories". Trabajo de Fin de Máster. Madrid: Universidad Autónoma de Madrid.

Barendregt, Henk P. and Erik Barendsen (2000). *Introduction to Lambda Calculus*. Available as PDF. Revised edition (Dec. 1998; Mar. 2000). Lecture notes. URL: `https://www.cse.chalmers.se/research/group/logic/TypesSS05/Extra/geuvers.pdf`.

Girard, Jean-Yves (1989). *Proofs and Types*. Trans. by Paul Taylor and Yves Lafont. Cambridge Tracts in Theoretical Computer Science 7. Translated and with appendices by Paul Taylor; typeset/appendix by Yves Lafont. Cambridge, UK: Cambridge University Press. ISBN: 9780521371810.

Selinger, Peter (2008). *Lecture Notes on the Lambda Calculus*. Lecture notes / arXiv preprint (revised versions available). Developed from courses taught 2001–2013. URL: `https://www.mathstat.dal.ca/~selinger/papers/lambdanotes.pdf`.

Sørensen, Morten Heine and Pawe Urzyczyn (2006). *Lectures on the Curry–Howard Isomorphism*. Vol. 149. Studies in Logic and the Foundations of Mathematics. Amsterdam & Boston: Elsevier / North-Holland. ISBN: 9780444520777.

Nederpelt, Rob and Herman Geuvers (2014). *Type Theory and Formal Proof: An Introduction*. Cambridge, UK: Cambridge University Press. ISBN: 9781107036505.